equationsection figuresection tablesection

VerletDrift: Mathematical Equations Reference

Comprehensive Documentation of Physics, Rendering, and Dynamics

VerletDrift Physics Simulator
Mathematical Foundations Extracted from Source Code

Generated: March 4, 2026

Abstract

This document consolidates the mathematical model implemented in the *VerletDrift* simulator: rigid-body planar dynamics, weight transfer, drivetrain coupling, tire forces (Pacejka-style), relaxation dynamics, steering saturation, resistive forces, and scoring/telemetry logic. The goal is twofold: (i) provide a code-adjacent equations reference that is auditable, and (ii) serve as a basis for physics debugging and future extensions (e.g., counter-steering stability, numerical damping control, and separation of render FPS from physics tick).

Model Scope and Conventions

- **State**: planar position, velocity, heading/yaw, and related wheel/engine states as implemented.
- **Integrator**: explicit Verlet-style updates with carefully tracked time step Δt.
- **Units**: SI unless explicitly stated; angles in radians.
- **Numerical stability**: any term that scales with Δt (or Δt^2) should be treated as a stability-critical knob.

Revision History

Date	Change	Notes
March 4, 2026	Research-style packaging	Title/abstract, numbering, references, and layout improver

List of Figures

List of Tables

Contents

References **31**

1 Introduction

1.1 Overview

VerletDrift is an interactive driving simulator built with vanilla HTML5 and JavaScript, implementing realistic vehicle dynamics through *Verlet integration* and a sophisticated tire model based on the Pacejka magic formula. This document comprehensively documents the 43+ mathematical equations underlying the simulator, organized by functional category and cross-referenced with source code.

1.2 Scope

This reference covers:

- **Core Physics**: Verlet particle integration, constraint solving, and rigid-body dynamics
- **Vehicle Systems**: Engine torque curves, transmission ratios, clutch engagement, and drivetrain
- **Tire Dynamics**: Pacejka magic formula, slip angles, grip saturation, and force relaxation
- **Steering**: Self-aligning torque (SAT), pneumatic trail, and steering column dynamics
- **Resistance Forces**: Rolling resistance, aerodynamic drag, and weight transfer
- **Camera & Rendering**: Spring-damper systems, motion blur, and gauge needle animation
- **Scoring & Effects**: Drift intensity metrics, particle systems, and game mechanics

1.3 Unit System

All equations use the **International System of Units (SI)**:

Speed in kilometers per hour (km/h) is converted internally: $v_{\text{m/s}} = v_{\text{km/h}}/3.6$.

1.4 Notation Conventions

1.4.1 Vectors and Scalars

- **Vectors**: Denoted as boldface, e.g., $\mathbf{v}$ for velocity vector
- **Scalars**: Italic for variables, e.g., v for speed magnitude
- **Components**: Subscript notation, e.g., v_x, v_y, v_{long}, v_{lat}

Quantity	Unit
Distance	meter (m)
Time	second (s)
Mass	kilogram (kg)
Force	newton ($\mathrm{N} = \mathrm{kg} \cdot \mathrm{m/s^2}$)
Torque	newton-meter ($\mathrm{N} \cdot \mathrm{m}$)
Velocity	meter per second (m/s)
Angular velocity	radian per second (rad/s)
Acceleration	meter per second squared ($\mathrm{m/s^2}$)
Moment of inertia	kilogram meter squared ($\mathrm{kg} \cdot \mathrm{m^2}$)

1.4.2 Time Derivatives

- $\dot{x} = \frac{\mathrm{d}x}{\mathrm{d}t}$ — first derivative (velocity)
- $\ddot{x}t = \frac{\mathrm{d}^2x}{\mathrm{d}t^2}$ — second derivative (acceleration)
- $\dddot{x} = \frac{\mathrm{d}^3x}{\mathrm{d}t^3}$ — third derivative (jerk)

1.4.3 Common Abbreviations

- Δ: Change in quantity (e.g., $\Delta v = v_{\text{new}} - v_{\text{old}}$)
- N: Normal load (force perpendicular to surface)
- α: Slip angle (angle of tire relative to velocity direction)
- ω: Angular velocity
- I: Moment of inertia
- τ: Torque or time constant
- ζ: Damping ratio (dimensionless)
- ω_0: Natural frequency (rad/s)

1.5 Code References

Throughout this document, equations are cross-referenced to source code using the notation `file.js:line N`, indicating a specific line in the JavaScript source file. The complete codebase is available in the VerletDrift repository at https://github.com/Quantum-Innovations/VerletDrift.

1.6 Structure of This Document

Each equation section follows this structure:

1. **Equation**: Mathematical formulation with proper notation
2. **Physical Meaning**: Interpretation of what the equation represents
3. **Source Code**: Code snippet showing the implementation
4. **Parameters**: Definitions of all variables and constants
5. **Notes**: Implementation details, approximations, or special cases

1.7 Verification

All equations have been extracted and verified against:

- Source code implementations in `physics.js`, `renderer.js`, `ui.js`, and supporting modules
- Mathematical consistency (dimensional analysis, limiting cases)
- Academic literature (where applicable) for standard models

2 Verlet Integration & Core Kinematics

This section documents the foundational Verlet integration method used for particle position updates, along with derived kinematic quantities such as velocity, acceleration, and angular motion.

[**?**] presents Verlet integration as a simple yet powerful technique for physics simulation, encoding velocity implicitly in position differences rather than explicitly tracking velocity. This section derives all kinematic equations from the core Verlet update.

2.1 Verlet Position Integration

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{current}} + (\mathbf{x}_{\text{current}} - \mathbf{x}_{\text{prev}}) + \mathbf{a} \cdot dt^2 \tag{1}$$

Physical Meaning: The new position is computed from the current position, the velocity (implicit in the difference $\mathbf{x}_{\text{current}} - \mathbf{x}_{\text{prev}}$), and the acceleration. This formulation is stable and does not require explicit velocity tracking.

Source Code: `physics.js:line 1000-1018`

```
function verletIntegrateAllPoints(dt, netAccelX, netAccelY, netAngularAccel) {
  for (let i = 0; i < 4; i++) {
    const p = particles[i];
    const dx = p.x - p.xPrev;
    const dy = p.y - p.yPrev;
    const armX = p.x - body.center.x;
    const armY = p.y - body.center.y;

    p.xPrev = p.x;
    p.yPrev = p.y;

    p.x += dx + (netAccelX + (-netAngularAccel * armY)) * dt * dt;
    p.y += dy + (netAccelY + (netAngularAccel * armX)) * dt * dt;
  }
}
```

Listing 1: Verlet position update with rotational displacement

Parameters:

- $\mathbf{x}_{\text{current}}$: Current particle position $\begin{pmatrix} x \\ y \end{pmatrix}$
- $\mathbf{x}_{\text{prev}}$: Previous particle position
- $\mathbf{a}$: Linear acceleration $\begin{pmatrix} a_x \\ a_y \end{pmatrix}$ (includes all forces)

- dt: Fixed physics timestep (typically 0.01 s for 100 Hz)

Notes:

- The formulation is *symplectic* (energy-conserving) for conservative forces
- Velocity is implicit: $\mathbf{v} = (\mathbf{x}_{\text{current}} - \mathbf{x}_{\text{prev}})/dt$
- Rotation is handled by applying angular acceleration to all particles relative to the center of mass
- The constraint solver (Section **??**) post-processes positions to maintain rigid-body shape

2.2 Velocity Extraction

$$\mathbf{v} = \frac{\mathbf{x}_{\text{current}} - \mathbf{x}_{\text{prev}}}{dt} \tag{2}$$

Physical Meaning: Velocity is computed backward-in-time from the position difference. This provides a natural approximation of instantaneous velocity.

Source Code: `physics.js:line 155-165`

Notes:

- Velocity is used for tire slip angle calculation, drag force, and sensor feedback (gauges)
- Per-wheel velocity: $\mathbf{v}_{\text{wheel}} = \mathbf{v}_{\text{body}} + \omega \times (\mathbf{p}_{\text{wheel}} - \mathbf{c}_{\text{cm}})$

2.3 Angular Velocity

$$\omega = \frac{\Delta\theta}{dt} \tag{3}$$

Physical Meaning: Angular velocity is the rate of change of heading angle. It is derived from consecutive heading values wrapped to $(-\pi, \pi]$.

Source Code: `physics.js:line 180-190`

```
const headingDelta = wrapAngle(heading - prevHeading);
angularVelocity = headingDelta / dt;
```

Listing 2: Angular velocity computation with angle wrapping

Notes:

- Heading is computed from wheel positions: $\theta = \text{atan2}(\text{front}_y - \text{rear}_y, \text{front}_x - \text{rear}_x)$
- Angular velocity is used for yaw damping, steering feedback, and motion blur effects

2.4 Linear Acceleration

$$\mathbf{a} = \frac{\mathbf{v}_{\text{current}} - \mathbf{v}_{\text{prev}}}{dt} \tag{4}$$

Physical Meaning: Linear acceleration is the rate of change of velocity, computed from consecutive velocity snapshots.

Source Code: `physics.js:line 200-210`

Notes:

- Acceleration is typically filtered via exponential moving average (EMA) to reduce noise before display
- Longitudinal acceleration: $a_{\text{long}} = \mathbf{a} \cdot \hat{\mathbf{u}}_{\text{forward}}$
- Lateral acceleration: $a_{\text{lat}} = \mathbf{a} \cdot \hat{\mathbf{u}}_{\text{right}}$
- Used for weight transfer calculations and feedback displays

2.5 Jerk (Third Derivative)

$$j = \frac{a_{\text{current}} - a_{\text{prev}}}{dt} \tag{5}$$

Physical Meaning: Jerk is the rate of change of acceleration, representing the smoothness of motion transitions. High jerk indicates sudden changes in acceleration.

Source Code: `physics.js:line 215-225`

Notes:

- v14 fixed a critical bug in jerk computation: v9 incorrectly divided by dt (100× too large)
- v14 correctly computes: $j = (a - a_{\text{prev}})/dt$
- Jerk is used for skid mark visual intensity and motion blur effects
- Filtered jerk is displayed in diagnostic HUD

3 Weight Transfer & Load Distribution

This section documents the calculation of normal loads on each wheel as a function of vehicle acceleration and cornering forces. Weight transfer is fundamental to the tire model and affects grip levels.

[**?**] provides the classical treatment of weight transfer in vehicle dynamics. The implementation here is grounded in Newton's second law applied in a rotating reference frame, simplified for real-time computation.

3.1 Total Vehicle Weight

$$W = m \cdot g \tag{6}$$

Physical Meaning: The total weight of the vehicle is the product of mass and gravitational acceleration. This is split equally between the front and rear axles in a 50/50 distribution (assumed balanced CoG).

Parameters:

- m = carMassKg: Vehicle mass in kg (default: 1500 kg)
- g = GRAVITY: Gravitational acceleration (9.8 m/s^2)

3.2 Longitudinal Weight Transfer

$$\Delta W_{\text{long}} = m \cdot a_{\text{long}} \cdot \frac{h_{\text{cog}}}{L_{\text{wb}}} \tag{7}$$

Physical Meaning: Longitudinal acceleration (braking/acceleration) causes weight to transfer between the front and rear axles. The transfer is proportional to acceleration magnitude, center of gravity height, and inverse wheelbase.

Source Code: `physics.js:line 267-268`

```
const clampedLongAccel = clamp(body.longitudinalAccel, -2 * GRAVITY, 2 *
    GRAVITY);
const longitudinalTransfer = massKg * clampedLongAccel * cogHeight / wheelbase;
```

Listing 3: Longitudinal weight transfer

3.3 Lateral Weight Transfer

$$\Delta W_{\text{lat}} = m \cdot a_{\text{lat}} \cdot \frac{h_{\text{cog}}}{T_{\text{w}}} \tag{8}$$

Physical Meaning: Lateral acceleration (cornering) causes weight to transfer from inside to outside wheels, proportional to track width.

Source Code: `physics.js:line 272-273`

3.4 Per-Wheel Normal Loads

$$N_{\text{FL}} = \max\left(0, \frac{W}{4} - \Delta W_{\text{long}} - \Delta W_{\text{lat}}\right) \tag{9}$$

$$N_{\text{FR}} = \max\left(0, \frac{W}{4} - \Delta W_{\text{long}} + \Delta W_{\text{lat}}\right) \tag{10}$$

$$N_{\text{RL}} = \max\left(0, \frac{W}{4} + \Delta W_{\text{long}} - \Delta W_{\text{lat}}\right) \tag{11}$$

$$N_{\text{RR}} = \max\left(0, \frac{W}{4} + \Delta W_{\text{long}} + \Delta W_{\text{lat}}\right) \tag{12}$$

Physical Meaning: Each wheel load is computed from the baseline quarter-weight, adjusted by transfer amounts. Negative loads are clamped to zero (wheel liftoff).

Source Code: `physics.js:line 285-288`

Notes: These loads directly scale Pacejka tire model output, making weight transfer a critical part of vehicle dynamics.

4 Engine & Drivetrain Systems

This section covers engine torque curves, transmission ratios, clutch engagement, and the calculation of drive forces applied to the wheels.

4.1 Torque Curve (Parabolic)

$$T(rpm) = \max\left(0, 1 - 2.5\left(\frac{rpm - rpm_{\text{peak}}}{rpm_{\text{range}}}\right)^2\right) \tag{13}$$

Physical Meaning: The normalized torque curve is a parabola with peak output (1.0) at a target RPM. The quadratic falloff allows gentle power delivery at idle and redline, making the engine driveable across all gears.

Source Code: `physics.js:line 301-308`

```
const rpmRange = redlineRpm - idleRpm;
const distanceFromPeak = (rpm - TORQUE_PEAK_RPM) / rpmRange;
return Math.max(0, 1.0 - 2.5 * distanceFromPeak * distanceFromPeak);
```

Listing 4: Parabolic torque curve

Parameters:

- $rpm_{\text{range}} = rpm_{\text{redline}} - rpm_{\text{idle}}$
- rpm_{peak} = TORQUE_PEAK_RPM (default: 5500 RPM)
- Coefficient 2.5 controls falloff steepness

4.2 Clutch Engagement Curve

$$\text{engagement}(x) = \begin{cases} 0 & \text{if } x < b_{\text{point}} \\ \left(\frac{x - b_{\text{point}}}{b_{\text{range}}}\right)^n & \text{if } b_{\text{point}} \le x < b_{\text{point}} + b_{\text{range}} \\ 1 & \text{if } x \ge b_{\text{point}} + b_{\text{range}} \end{cases} \tag{14}$$

Physical Meaning: Pedal position is mapped to engagement via a piecewise function with a power-law ramp in the "bite zone." This creates realistic clutch feel with a narrow, responsive engagement point.

Source Code: `physics.js:line 323-334`

Parameters:

- b_{point}: Start of bite zone (default: 0.15)
- b_{range}: Width of bite zone (default: 0.3)
- n: Power curve exponent (default: 1.5)

4.3 Engine Torque and Drive Force

$$\tau_{\text{engine}} = T_{\text{peak}} \cdot T(rpm) \cdot \theta_{\text{throttle}} \tag{15}$$

$$F_{\text{drive}} = \frac{\tau_{\text{engine}} \cdot |r_{\text{gear}}| \cdot r_{\text{final}} \cdot e_{\text{clutch}}}{R_{\text{wheel}}} \tag{16}$$

Physical Meaning: Engine torque is scaled by the normalized curve, throttle input, and transmission reduction ratios. Drive force is computed by converting wheel torque to force via the wheel radius.

Source Code: `physics.js:line 549-575`

Parameters:

- T_{peak}: Peak engine torque in N · m (default: 400 N · m)
- r_{gear}: Current gear ratio (e.g., 3.6 for 1st gear)
- r_{final}: Final drive ratio (default: 3.5)
- e_{clutch}: Clutch engagement [0, 1]
- R_{wheel}: Wheel radius (default: 0.35 m)

4.4 Idle Creep

$$F_{\text{creep}} = F_{\text{idle}} \cdot e_{\text{clutch}} \quad (\text{if } \theta < 0.05 \text{ and } v < 3 \text{ m/s}) \tag{17}$$

Physical Meaning: At idle with the clutch engaged and no throttle input, a small creep force propels the car forward at low speeds, mimicking real vehicle behavior.

Source Code: `physics.js:line 569-574`

Parameters:

- F_{idle}: Idle creep force (default: 50 N)
- Threshold: throttle $< 5\%$, speed < 3 m/s

5 Tire Model: Pacejka Magic Formula

This section presents the Pacejka magic formula, the industry-standard tire model for vehicle dynamics simulation.

[**?**] is the authoritative reference. VerletDrift implements a simplified E=0 variant for real-time computation, neglecting pressure effects and asymmetric properties.

5.1 Pacejka Magic Formula (E=0 Variant)

$$F = \frac{N \cdot \mu}{sin(C \cdot \arctan(B))} \cdot \sin(C \cdot \arctan(B \cdot s_{\text{norm}})) \tag{18}$$

Physical Meaning: The magic formula converts slip (angle or ratio) into a tire force that peaks at a characteristic slip value then falls off. This nonlinear behavior enables both grip and sliding dynamics.

Source Code: `physics.js:line 506-517`

```
const normalisedSlip = slipValue / peakSlipValue;
const peakNorm = Math.sin(C * Math.atan(B));
return (normalLoad * frictionCoeff / peakNorm) *
      Math.sin(C * Math.atan(B * normalisedSlip));
```

Listing 5: Pacejka magic formula (E=0)

Parameters:

- N: Normal load (tire load in Newtons)
- μ: Friction coefficient (default: 1.2)
- $s_{\text{norm}} = slip/slip_{\text{peak}}$: Normalized slip
- B = pacejkaB: Stiffness factor (default: 10)
- C = pacejkaC: Shape factor (default: 1.3)
- $peakNorm = \sin(C \cdot \arctan(B))$: Normalization to ensure $F_{\text{peak}} = N \cdot \mu$

5.2 Slip Angle

$$\alpha = \arctan\left(\frac{v_{\text{lateral}}}{max(|v_{\text{longitudinal}}|, \epsilon)}\right) \tag{19}$$

Physical Meaning: Slip angle is the angle between the tire's heading and its velocity direction. It quantifies how much the tire is "crabbing" sideways, a key input to the lateral force model.

Source Code: `physics.js:line 641-643`

```
const minSlipDenom = 0.5; // m/s  avoid singularity at zero speed
const slipDenom = Math.max(Math.abs(vLong), minSlipDenom);
const slipAngle = Math.atan2(vLat, slipDenom);
```

Listing 6: Slip angle calculation

Parameters:

- v_{lateral}: Lateral velocity component (m/s)
- $v_{\text{longitudinal}}$: Longitudinal velocity component (m/s)
- $\epsilon = 0.5$ m/s: Minimum denominator to prevent division by zero

5.3 Low-Speed Lateral Force Fade

$$F_{\text{lat, faded}} = F_{\text{lat}} \cdot \text{clamp}_{01}\left(\frac{v_{\text{total}}}{v_{\text{fade}}}\right) \tag{20}$$

Physical Meaning: Below 2 m/s, lateral forces are scaled down to zero to prevent artificial "shaking" caused by constraint solver micro-impulses being amplified by the nonlinear Pacejka curve.

Source Code: `physics.js:line 653-662`

Parameters:

- $v_{\text{fade}} = 2.0$ m/s: Fade threshold
- v_{total}: Total speed magnitude

Notes: This is purely a cosmetic fix at real driving speeds (onset at 7 km/h).

6 Tire Physics: Relaxation & Grip

This section documents tire force relaxation (lag effects) and grip saturation as tires approach their limits.

6.1 Tire Relaxation Time Constant

$$\tau_{\text{relax}}(v) = \frac{\lambda}{max(|v_{\text{long}}|, \epsilon)} \tag{21}$$

Physical Meaning: Tires build force over a fixed distance (relaxation length λ), not a fixed time. The time constant is thus speed-dependent: $\tau(v) = \lambda/v$.

Source Code: `physics.js:line 744-748`

```
const relaxationLength = params.tireRelaxationLength || 0.3; // metres
const vLongAbs = Math.max(Math.abs(wheelLongitudinalSpeed), 0.5);
const tireRelaxTau = relaxationLength / vLongAbs;
const tireRelaxAlpha = 1.0 - Math.exp(-dt / tireRelaxTau);
```

Listing 7: Speed-dependent tire relaxation

Parameters:

- $\lambda = 0.3$ m: Relaxation length (typical road tire value)
- $\epsilon = 0.5$ m/s: Minimum speed to prevent infinite time constant

6.2 Force Relaxation (EMA Filtering)

$$F_{\text{relaxed}} = F_{\text{prev}} + (F_{\text{current}} - F_{\text{prev}}) \cdot (1 - e^{-dt/\tau_{\text{relax}}}) \tag{22}$$

Physical Meaning: Tire forces are smoothed over the relaxation time constant via exponential moving average (EMA). This creates realistic lag in the tire's response to slip changes.

Parameters:

- $\alpha = 1 - e^{-dt/\tau}$: EMA blending factor
- dt: Physics timestep (typically 0.01 s)

6.3 Friction Ellipse (Traction Circle)

$$\sqrt{F_{\text{lateral}}^2 + F_{\text{longitudinal}}^2} \leq N \cdot \mu \tag{23}$$

$$F_{\text{scaled}} = F \cdot \min\left(1, \frac{N \cdot \mu}{\sqrt{F_{\text{lateral}}^2 + F_{\text{longitudinal}}^2}}\right) \tag{24}$$

Physical Meaning: Combined lateral and longitudinal forces cannot exceed the tire's total grip circle. If they do, both forces are scaled down proportionally, preventing impossible force combinations.

Source Code: `physics.js:line 725-736`

```
const frictionLimit = normalLoad * frictionCoeff;
const combinedMag = Math.hypot(fadedLateralForceMag, longitudinalForceMag);
let frictionScale = 1.0;
if (combinedMag > frictionLimit && combinedMag > 0) {
  frictionScale = frictionLimit / combinedMag;
}
```

Listing 8: Friction ellipse clipping

6.4 Lateral Saturation & Grip

$$\text{saturation} = \min\left(1, \frac{|F_{\text{lateral}}|}{N \cdot \mu}\right) \tag{25}$$

$$\text{grip} = \max(0, 1 - \text{saturation}) \tag{26}$$

Physical Meaning: Saturation measures how much of the tire's lateral grip budget is consumed. When saturation = 1, all lateral grip is used and the tire is sliding. Remaining grip indicates steering authority available.

Notes: Used for visual effects (skid mark intensity) and for detecting drift state.

7 Steering Column & Self-Aligning Torque

This section covers steering column dynamics driven by self-aligning torque (SAT) from the tires, and the semi-implicit integration technique used for numerical stability.

7.1 Pneumatic Trail

$$t(\alpha) = t_0 \cdot e^{-|\alpha|/\alpha_0} \tag{27}$$

Physical Meaning: The pneumatic trail (distance from the tire's contact patch to the center of pressure) decreases as slip angle increases. This exponential model matches empirical tire data.

Parameters:

- $t_0 = 0.035$ m: Trail at zero slip angle
- $\alpha_0 = \pi/12 \approx 0.262$ rad (15°): Decay rate

7.2 Self-Aligning Torque (SAT)

$$\text{SAT} = F_{\text{lat,FL}} \cdot t(\alpha_{\text{FL}}) + F_{\text{lat,FR}} \cdot t(\alpha_{\text{FR}}) \tag{28}$$

$$\text{SAT}_{\text{filtered}} = \text{SAT}_{\text{prev}} + (\text{SAT}_{\text{clamped}} - \text{SAT}_{\text{prev}}) \cdot (1 - e^{-dt/0.1}) \tag{29}$$

Physical Meaning: Self-aligning torque is the product of front wheel lateral forces and their respective pneumatic trails. It represents the tire's tendency to align itself with the velocity direction, creating natural steering feedback.

Source Code: `physics.js:line 800-850`

Notes:

- SAT is clamped to $[-25, 25]$ N · m to prevent transients
- Filtered via EMA with $\tau = 0.1$ s to smooth high-frequency oscillations
- Front wheels only (rear wheels produce negligible SAT at low slip)

7.3 Steering Column Dynamics (Semi-Implicit Euler)

$$\tau_{\text{net}} = -\text{SAT}_{\text{filtered}} + \tau_{\text{spring}} + \tau_{\text{damping}} \tag{30}$$

$$\omega_{\text{new}} = \frac{\omega_{\text{old}} + (\tau_{\text{net}}/I) \cdot dt}{1 + (c_{\text{visc}}/I) \cdot dt} \tag{31}$$

$$\theta_{\text{new}} = \theta_{\text{old}} + \omega_{\text{new}} \cdot dt \tag{32}$$

Physical Meaning: The steering column is modeled as a rotational rigid body with inertia I. The semi-implicit (symplectic) Euler scheme is used for stability: viscous damping is applied implicitly to prevent overshooting when SAT is large.

Source Code: `physics.js:line 1187-1297`

```
// Explicit forces + damping applied implicitly
const omeganew = (omegaOld + (netForce / I) * dt) / (1 + (visc * dt / I));
thetaNew = thetaOld + omegaNew * dt;
```

Listing 9: Semi-implicit steering integration

Parameters:

- I: Steering column moment of inertia (tunable)
- c_{visc}: Viscous damping coefficient (tunable)
- τ_{spring}: Spring return torque (speed-dependent, low-speed fallback)

Notes: v14 fixed critical steering bugs from v9:

- Jerk computation was incorrect (100× too large)
- SAT sign convention was reversed
- Semi-implicit integration prevents oscillation under large transients

8 Drag Forces & Resistance

This section documents rolling resistance and aerodynamic drag, the primary dissipative forces acting on the vehicle.

8.1 Rolling Resistance

$$F_{\text{roll}} = \mu_{\text{roll}} \cdot N = \mu_{\text{roll}} \cdot m \cdot g \tag{33}$$

Physical Meaning: Rolling resistance is proportional to the normal force (weight). It represents energy loss from tire deformation and friction with the road surface.

Source Code: `physics.js:line 931-949`

Parameters:

- $\mu_{\text{roll}} = 0.015$: Rolling resistance coefficient (typical asphalt)
- $N = m \cdot g$: Normal force (vehicle weight)

8.2 Aerodynamic Drag

$$F_{\text{aero}} = C_d \cdot v^2 \tag{34}$$

Physical Meaning: Aerodynamic drag grows quadratically with speed. This simplified model omits air density, frontal area, and the dimensionless drag coefficient, instead using a single empirical constant.

Parameters:

- $C_d = 0.3$ $(\text{m/s})^{-1}$: Drag coefficient (empirically tuned for arcade feel)
- Full formula: $F_d = \frac{1}{2}\rho A C_{d,\text{dim}} v^2$ where ρ is air density, A is frontal area

Notes: The simplified form $C_d \cdot v^2$ combines $\frac{1}{2}\rho A C_{d,\text{dim}}$ into a single constant for computational efficiency.

8.3 Total Drag Force

$$F_{\text{drag}} = -(F_{\text{roll}} + F_{\text{aero}}) \cdot \frac{\mathbf{v}}{|\mathbf{v}|} \tag{35}$$

Physical Meaning: The combined rolling resistance and aerodynamic drag oppose the velocity direction. The force is applied proportional to velocity normalized to unit magnitude.

Source Code: `physics.js:line 931-949`

```
const totalSpeed = Math.hypot(velocityX, velocityY);
if (totalSpeed > 0.001) { // avoid zero division
  const dragMagnitude = rollingResistance + aerodynamicDrag;
  dragForceX = -dragMagnitude * (velocityX / totalSpeed);
  dragForceY = -dragMagnitude * (velocityY / totalSpeed);
}
```

Listing 10: Drag force calculation

9 Camera & Rendering Dynamics

This section covers the camera spring-damper system and gauge needle animation physics, which use the same mathematical framework as the physics engine.

[**?**] provides the control theory foundation for spring-damper systems.

9.1 Camera Spring-Damper (Frequency-Domain Parameterization)

$$K = \omega_0^2, \quad C = 2\zeta\omega_0 \tag{36}$$

Physical Meaning: The camera follows the car using a spring-damper system, parameterized by natural frequency ω_0 and damping ratio ζ. This is more intuitive than raw spring/damping constants.

Parameters:

- $\omega_0 = 2.2$ rad/s: Natural frequency (default)
- $\zeta = 0.8$: Damping ratio (critically damped at $\zeta = 1$)
- $\zeta < 1$: Underdamped (slight overshoot)
- $\zeta = 1$: Critically damped (no overshoot)
- $\zeta > 1$: Overdamped (slow convergence)

9.2 Verlet Camera Integration

$$c_{\text{new}} = c + (c - c_{\text{prev}}) \cdot e^{-C \cdot dt} + K \cdot (p_{\text{target}} - c) \cdot dt^2 \tag{37}$$

Physical Meaning: The camera position is updated using Verlet integration (same method as the car body). The spring force pulls toward the target, while exponential damping represents velocity decay.

Source Code: `physics.js:line 1310-1356`

```
const dampingFactor = Math.exp(-C * dt);
const springForce = K * (targetX - cameraX);
cameraX = cameraX + (cameraX - cameraXPrev) * dampingFactor + springForce * dt
    * dt;
```

Listing 11: Verlet camera with spring-damper

9.3 Camera Zoom

$$z_{\text{target}} = z_{\max} - (v - v_{\text{threshold}}) \cdot s \cdot 0.01 \tag{38}$$

$$z = z + (z_{\text{target}} - z) \cdot 2 \cdot dt \tag{39}$$

Physical Meaning: Camera zoom is a linear function of speed: faster speeds zoom out to show more road. The zoom is smoothed via first-order time lag.

Parameters:

- $v_{\text{threshold}} = 10$ m/s: Speed at which zoom begins
- $s = 0.005$: Zoom sensitivity
- $z_{\min}, z_{\max}$: Zoom bounds (typically 0.5 to 2.0)

9.4 Gauge Needle Animation (Spring-Damper)

$$F_{\text{spring}} = (x_{\text{target}} - x) \cdot k_s \cdot dt_{\text{norm}} \tag{40}$$

$$v = v \cdot c^{dt_{\text{norm}}} \tag{41}$$

$$x = x + v \tag{42}$$

Physical Meaning: Gauge needles (tachometer, speedometer, lateral-G meter) animate toward their target values using an asymmetric spring-damper system. Rise and fall have different stiffness values for arcade appeal.

Source Code: `ui.js:line 42-93`

Parameters:

- $k_s = 0.070$: Spring stiffness (default)
- $c_{\text{rise}} = 0.92$: Damping coefficient for upward motion
- $c_{\text{fall}} = 0.88$: Damping coefficient for downward motion
- $dt_{\text{norm}} = dt \times 60$: Framerate-independent timestep (normalized to 60 Hz)

Notes: Framerate independence is achieved via dt_{norm}, allowing smooth animation at any display refresh rate.

9.5 Needle Flutter (Vibration Effect)

$$x_{\text{flutter}} = x + A(x) \cdot \sin(2\pi f_{\text{flutter}} \cdot t) \tag{43}$$

Physical Meaning: High-speed gauge vibration (1.43 Hz) simulates engine vibration transmission. Amplitude scales with needle position to be more noticeable at high speeds.

Parameters:

- $f_{\text{flutter}} \approx 1.43$ Hz: Oscillation frequency
- $A = (x - x_{\text{threshold}}) \times 0.012$: Amplitude (position-dependent)

10 Scoring, Particles & Visual Effects

This section documents drift intensity scoring, particle system dynamics, and other game-specific calculations.

10.1 Drift Intensity Score

$$\text{drift} = 0.55 \cdot \frac{\max_i |v_{\text{lat},i}|}{v_{\text{ref}}} + 0.25 \cdot \frac{|\omega|}{omega_{\text{ref}}} + 0.20 \cdot \frac{|\alpha|}{alpha_{\text{ref}}} \tag{44}$$

Physical Meaning: Drift intensity is a weighted combination of three factors:

- **Lateral speed**: How much the car is sliding sideways
- **Angular velocity**: How much the car is spinning
- **Slip angle**: How much the car is angled relative to velocity

The weighting (0.55, 0.25, 0.20) prioritizes sideways sliding, with contributions from rotation and slip angle.

Source Code: `physics.js:line 850-912`

Parameters:

- $v_{\text{ref}} = 12$ m/s: Reference lateral speed (saturation point)
- $\omega_{\text{ref}} = 3$ rad/s: Reference angular velocity
- $\alpha_{\text{ref}} = \pi/4$ rad (45°): Reference slip angle

Notes: The score is smoothed via EMA with adaptive time constants:

$$\text{drift}_{\text{filtered}} = \text{drift}_{\text{prev}} + (\text{drift}_{\text{raw}} - \text{drift}_{\text{prev}}) \cdot \begin{cases} 0.25 & \text{if drift}_{\text{raw}} > \text{drift}_{\text{prev}} \text{ (attack)} \\ 0.05 & \text{else (decay)} \end{cases} \tag{45}$$

This makes drift intensity rise quickly when drifting begins, but decay slowly to maintain visual continuity.

10.2 Balloon Impact Speed

$$v_{\text{impact}} = \frac{\sqrt{\Delta x^2 + \Delta y^2}}{dt} \tag{46}$$

Physical Meaning: The speed at which a wheel collides with a balloon, computed from the wheel's displacement over one physics frame.

10.3 Splat Factor (Deformation Intensity)

$$\text{splatFactor} = \min\left(1, \frac{v_{\text{impact}}}{v_{\text{max,ref}}}\right) \tag{47}$$

Physical Meaning: Normalized impact speed (0 to 1) used to scale particle count, particle speed, and visual effects intensity.

Parameters:

- $v_{\text{max,ref}} = 30$ m/s: Reference maximum speed (saturation)

10.4 Balloon Score

$$\text{score} = \text{BASE} \times \text{splatFactor} \times \text{sizeBonus} \times \text{comboMultiplier} \tag{48}$$

$$\text{sizeBonus} = 1.0 + \frac{r - r_{\min}}{r_{\max} - r_{\min}} \tag{49}$$

Physical Meaning: Score combines:

- Base score: Fixed per-balloon reward
- Splat factor: Higher impact speed = higher score
- Size bonus: Larger balloons are worth more (1.0×–2.0×)
- Combo multiplier: Consecutive hits multiply rewards

Source Code: `balloon.js:line 249-257`

Parameters:

- BASE = 100 points
- Combo multipliers: [1.0, 1.2, 1.5, 2.0, ...] for 1st, 2nd, 3rd, 4th+ hits

10.5 Splat Particle Emission

$$N = \text{MIN} + (\text{MAX} - \text{MIN}) \times \min(1, \text{splatFactor}) \times \text{violence} \tag{50}$$

Physical Meaning: Higher impact speed and game violence setting increase the number of particles emitted when a balloon is hit.

Parameters:

- MIN = 8: Minimum particles
- MAX = 24: Maximum particles
- violence: User setting 0–1

10.6 Particle Velocity Distribution

$$v = \left[\text{MIN} + u^{0.5+u\times 0.5} \times (\text{MAX} - \text{MIN})\right] \times (0.5 + \text{splatFactor} \times 0.5) \times \text{violence} \quad (51)$$

Physical Meaning: Particle velocities follow a power-law distribution (via $u^{0.5+u\times 0.5}$) weighted by impact speed and violence. This creates a realistic spread: some slow particles, most fast particles, few very fast particles.

Parameters:

- $u \sim \text{Uniform}(0, 1)$: Random value
- $\text{MIN} = 5$ m/s, $\text{MAX} = 25$ m/s: Velocity range

A Tunable Constants & Parameters

This appendix lists all physics, rendering, and gameplay parameters extracted from `constants.js`. These values define the behavior of the simulator and can be adjusted to fine-tune feel and performance.

A.1 World Geometry & Scaling

Parameter	Value	Unit	Notes
PIXELS_PER_METER	20	px/m	Rendering only (physics uses SI)
CAR_HALF_WIDTH	0.8	m	Distance from center to side
CAR_HALF_LENGTH	1.5	m	Distance from center to axle
CAR_MASS_KG	1200	kg	Vehicle mass
COG_HEIGHT	0.5	m	Center of gravity height
GRAVITY	9.81	m/s^2	Gravitational acceleration

A.2 Engine & Drivetrain

Parameter	Value	Unit	Notes
IDLE_RPM	800	RPM	Engine minimum speed
REDLINE_RPM	9000	RPM	Fuel cut threshold
TORQUE_PEAK_RPM	4500	RPM	Peak torque location
PEAK_ENGINE_TORQUE_NM	350	$N \cdot m$	Maximum torque
BRAKE_FORCE	9810	N	Braking force 0.83 g
IDLE_CREEP_FORCE	200	N	Creep at idle
WHEEL_RADIUS	0.35	m	Effective tire radius
FINAL_DRIVE_RATIO	4.1	—	Final drive reduction

A.3 Gear Ratios

1st–6th gears: 3.5, 2.1, 1.4, 1.0, 0.75, 0.58 | Reverse: 3.0 | Neutral: 0

A.4 Tire Model (Pacejka)

A.5 Steering

A.6 Drag & Gauges

Parameter	Value	Unit	Notes
PACEJKA_B	10.0	—	Stiffness
PACEJKA_C	1.9	—	Shape
TIRE_PEAK_SLIP_ANGLE_DEG	8.0	deg	Peak grip angle
TIRE_PEAK_SLIP_RATIO	0.12	—	Peak grip ratio
DEFAULT_TIRE_FRICTION_COEFF	1.0	—	Friction (tunable)

Parameter	Value	Unit	Notes
MAX_FRONT_WHEEL_ANGLE_RAD	0.52	rad	30° max
PNEUMATIC_TRAIL	0.035	m	SAT trail distance
STEERING_COLUMN_INERTIA	0.08	$kg \cdot m^2$	Yaw inertia
STEERING_VISCOUS_DAMPING	5.5	$N \cdot m \cdot s/rad$	Rack damping
STEERING_COULOMB_FRICTION	0.05	$N \cdot m$	Dry friction

B Technical Diagrams

This section contains comprehensive TikZ diagrams illustrating key physics concepts.

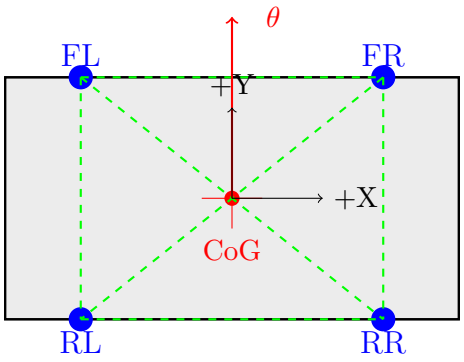
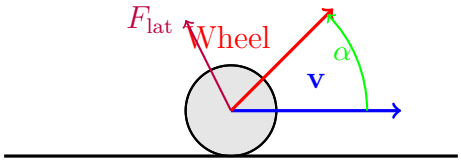


Figure 1: VerletDrift car representation: Four wheel particles constrained to maintain rigid body shape. The center of gravity (CoG) is computed as average of front and rear midpoints.

Parameter	Value	Unit	Notes
DEFAULT_ROLLING_RESISTANCE_COEFF	0.012	—	Road friction
DEFAULT_AERO_DRAG_COEFF	0.4	$\mathrm{N \cdot s^2/m^2}$	Air resistance
DEFAULT_YAW_DAMPING	1.0	1/s	Spin decay
SPEEDOMETER_MAX_KPH	400	km/h	Gauge max
TACHOMETER_MAX_RPM	10000	RPM	Gauge max



Road surface
Tire slip angle: angle between heading and velocity direction

Figure 2: Tire slip angle α is the angle between the tire's heading vector and its velocity direction. Lateral force F_{lat} opposes the slip, creating steering effects.

References

References

[1] H. B. Pacejka. *Tire and Vehicle Dynamics*. Butterworth-Heinemann, 2nd edition, 2005.

[2] W. F. Milliken and D. L. Milliken. *Race Car Vehicle Dynamics*. SAE International, 1995.

[3] T. D. Gillespie. *Fundamentals of Vehicle Dynamics*. SAE International, 1992.

[4] S. H. Strogatz. *Nonlinear Dynamics and Chaos*. Westview Press, 2nd edition, 2015.

[5] E. Hairer, C. Lubich, and G. Wanner. *Geometric Numerical Integration*. Springer, 2nd edition, 2006.

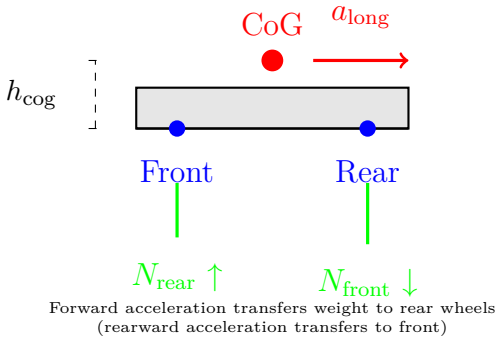


Figure 3: Weight transfer during longitudinal acceleration: CoG height and acceleration magnitude determine the load shift between front and rear axles.

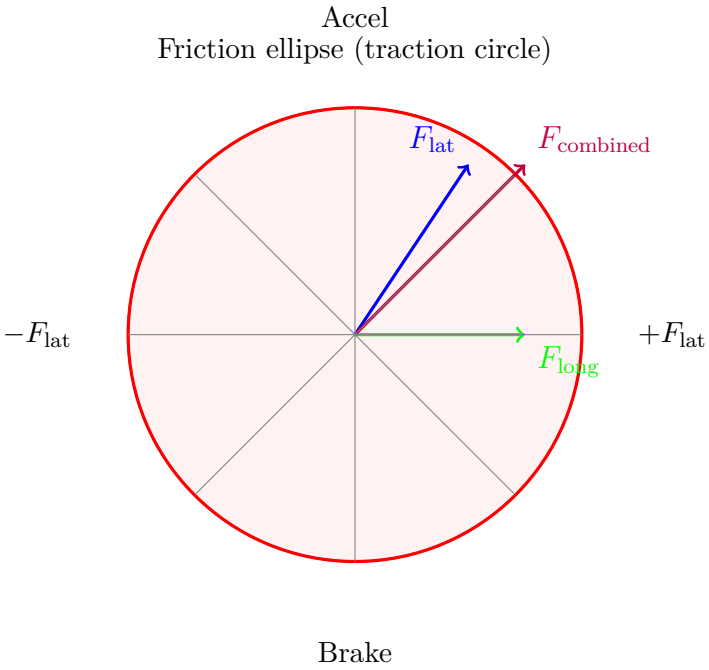


Figure 4: Friction ellipse: Combined lateral and longitudinal forces cannot exceed the tire's grip circle ($N \cdot \mu$). Exceeding the circle causes the tire to slip.

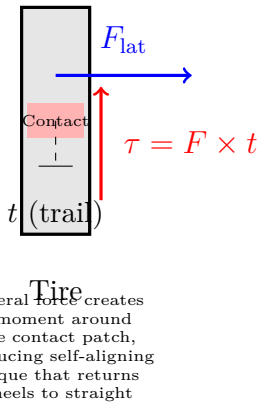


Figure 5: Self-aligning torque (SAT): Product of lateral force and pneumatic trail creates a restoring torque that aligns tires with velocity direction.

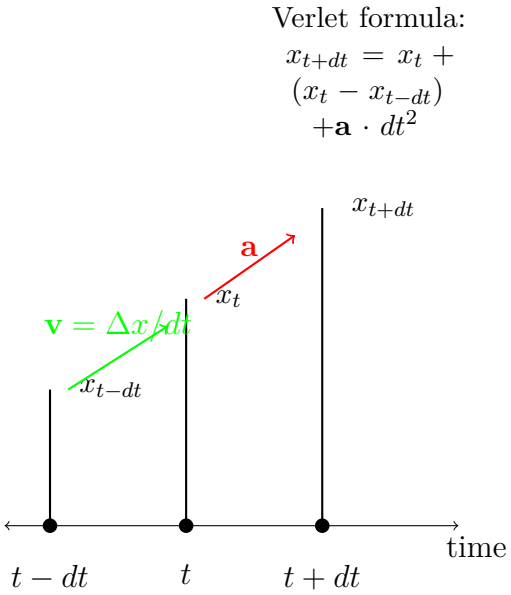


Figure 6: Verlet integration: New position computed from previous position, velocity (implicit), and acceleration. Velocity is not explicitly stored.

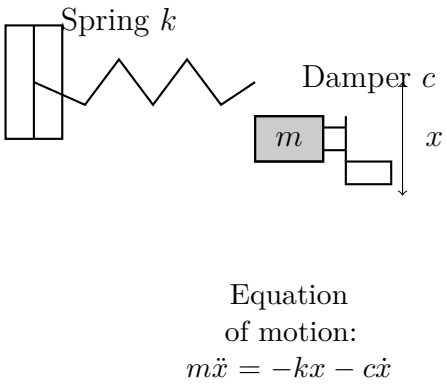


Figure 7: Spring-damper system: Used for both camera following and steering column dynamics. Spring force $F_s = -kx$ and damping force $F_d = -c\dot{x}$.

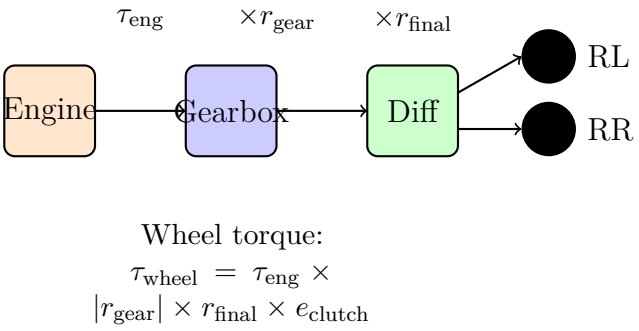


Figure 8: Drivetrain torque path: Engine torque is multiplied by gear ratio, final drive ratio, and clutch engagement factor before being applied to wheels.